NLP ДЗ №1: разбиение на 4 человек и контракты

Oct 9, 2026 · @Diana

Цель и общая схема

Сервис `POST /detect` превращается в конвейер из трёх модулей, которые собирает четвёртый участник, а пятый делает эталон и измеряет качество. Каждый модуль живёт в своём файле, поэтому конфликтов в git почти не будет.

#	Модуль (файл)	Что делает	Владелец
1	`law_matcher.py`	по тексту названия закона находит `law_id`	Участник 1
2	`link_extractor.py`	находит в тексте ссылки и режет их на уровни	Участник 2
3	`number_parser.py`	разбирает номера и разворачивает перечисления в отдельные `LawLink`	Участник 3
4	`main.py`, `Dockerfile`, `requirements.txt`	собирает конвейер, Docker, README, GitHub, CI	Участник 4
5	`tests/`, `eval/`	эталонная разметка, юнит-тесты, метрики, разбор ошибок	Участник 5

Порядок данных: текст → `extract_links` → список `RawLink` → для каждого `find_law` и `expand` → список `LawLink` → ответ.

Контракты между модулями

Всё, на чём модули стыкуются, — два типа данных и три функции. Их нужно зафиксировать в одном файле `contracts.py` в первый же день, до начала настоящей работы. Дальше файл меняется только по договорённости всех четырёх.

```
# contracts.py — эталонный эскиз, можно уточнять при обсуждении
from dataclasses import dataclass
from typing import Optional

@dataclass
class RawLink:
    """Одна найденная в тексте ссылка, значения — сырые строки как в
тексте."""
    start: int                          # позиция начала во входном тексте
    end: int                            # позиция конца (не включая)
    article: Optional[str] = None       # "374", "30.1"
    point: Optional[str] = None         # "1", "3.345, 23"  (пункт или часть)
    subpoint: Optional[str] = None      # "1, 2, 3", "а, б и с"
    law_text: Optional[str] = None      # фрагмент с названием закона, как в
тексте

# Выход сервиса — прежний LawLink из main.py (law_id, article, point_article,
subpoint_article)
```

Функция	Вход	Выход	Модуль
`extract_links(text)`	весь текст запроса	`list[RawLink]` в порядке появления в тексте	`link_extractor`
`find_law(law_text)`	строка с названием закона	`int` или `None`, если закон не найден	`law_matcher`
`expand(raw, law_id)`	`RawLink` и найденный `law_id`	`list[LawLink]`, по одному объекту на каждую комбинацию	`number_parser`

Общие правила: никаких исключений наружу (непонятный ввод даёт `None` или пустой список), без глобального состояния, типы указаны в сигнатурах. Индекс алиасов строится один раз при старте сервиса.

Участник 1. Сопоставление закона (`law_matcher.py`)

Задача: превратить «уголовного кодекса», «УК РФ» или «Закон "О полиции"» в `law_id` из `law_aliases.json`. В файле 1055 законов и 18 724 варианта написания, без учёта склонений.

- Строит индекс при старте: нормализация и лемматизация алиасов (регистр, кавычки `«»` и `"`, «№», пробелы), словарь «нормализованный алиас → `law_id`».
- При запросе нормализует `law_text` так же и ищет совпадение. Если подходят несколько алиасов, берёт самый длинный.
- Разбирает трудные случаи: «ФЗ №20-456», название в кавычках без типа акта, «Конституции Российской Федерации» в разных падежах.
- Для неизвестного закона («рандомный кодекс») возвращает `None`, а не ближайшее похожее: лучше пропустить, чем ошибиться.

Готово, когда: `find_law` проходит юнит-тесты на всех названиях законов из тестового текста README, а индекс строится за разумное время старта (цель уточнить в команде).

Участник 2. Поиск ссылок (`link_extractor.py`)

Задача: найти в тексте границы ссылки вида «подпункт → пункт → статья → закон» и разложить её на уровни. Значения уровней остаются сырыми строками: разбирать их — работа участника 3.

- Сокращения и полные слова в разных падежах: `ст.` / `статье` / `статьи`, `п.` / `пункта` / `пункте`, `пп.` / `подпункту` / `подпунктах`. «Часть» считается тем же уровнем, что и пункт (`point`).
- Два порядка записи: «пп. 1 п. 2 ст. 3 НК РФ» и обратный «статья 17 Конституции», «в пункте 1 статьи 34».
- Опечатки и шум из тестового текста: «пунта», «п. р», двойные пробелы, «и далее» после закона.
- Ссылка без закона («часть 3, ст. 30.1» без названия) тоже находится, `law_text` в неё — `None`.
- Граница `law_text`: вырезает только текст, который относится к названию («УК РФ», «Закон "О полиции"»), и не захватывает соседние слова.

Готово, когда: `extract_links` находит все ссылки тестового текста README с правильными границами и сырыми значениями, по эталону участника 5.

Участник 3. Разбор номеров (`number_parser.py`)

Задача: из одного `RawLink` сделать список `LawLink`, по одному на каждый кусок текста закона. Пример из README: `пп. 1, 2, 3 п. 2 ст. 3 НК РФ` даёт три объекта, которые отличаются только полем `subpoint_article`.

- Разбирает значения вида `12`, `4.6`, `а`, `43.2-6`, `1, 2, 3`, `5 и 6`, `а, б и с`, `3.345, 23`.
- Делает декартово произведение уровней: «пп. 4, 5, 6 и 8 п. 1 ст. 14» — 4 объекта, «пп. а, б п. 3, 4 ст. 5» — 4 объекта.
- Удаляет дубли и сохраняет порядок перечисления.
- Пустой уровень (`None`) остаётся `None`, а не превращается в пустую строку.
- Диапазоны (`43.2-6`) и точечные номера с точкой (`4.6`, `3.345`) не разбиваются и проходят как одна строка (решение см. в открытых вопросах).

Готово, когда: `expand` проходит табличные юнит-тесты (вход → ожидаемый список), не завися от остальных модулей. Работать можно с первого дня на вручную собранных `RawLink`.

Участник 4. Сервис и инфраструктура (`main.py`, `Dockerfile`)

Задача: собрать три модуля в работающий сервис и довести его до `http://localhost:8978/detect` в Docker.

- В первый день делает «скелет»: `contracts.py` и заглушки трёх функций, чтобы остальные не ждали друг друга.
- Функция `detect_links(text)` связывает `extract_links` → `find_law` → `expand` и вызывается из `/detect`. Индекс алиасов строится в `lifespan`, а не на каждый запрос.
- Обновляет `requirements.txt` (лемматизатор и другие зависимости) и `Dockerfile`: копирует все новые файлы, не забывая `law_aliases.json`.
- Проверяет сборку образа и запрос `curl` из README. В первую очередь смотрит на время старта и память: база весит 4,3 МБ.
- Ведёт публичный репозиторий на GitHub: ветки, слияние, README с инструкцией запуска.

Готово, когда: `docker build` и `docker run` проходят с нуля, `curl` на `localhost:8978/detect` возвращает правильный JSON.

Участник 5. Эталон, тесты и метрики (`tests/`, `eval/`)

Задача: дать команде объективную оценку, чтобы не проверять результат на глаз. Работа не зависит от кода остальных, поэтому начинается в первый день.

- Размечает эталон: тестовый текст из README (около 30 ссылок) плюс дополнительные примеры: перечисления, диапазоны, опечатки, обратный порядок, неизвестные законы. Формат — текст и ожидаемый список `LawLink`.
- Пишет скрипт оценки: precision, recall и F1 по кортежам `(law_id, article, point, subpoint)`, отдельно по каждому уровню.
- Пишет юнит-тесты на контракты между модулями и утилиту показа ошибок: какой текст, что ожидали, что получили.
- Разбирает ошибки после каждого запуска и отдаёт владельцу модуля, где ошибка возникла.
- После готовности эталона помогает участнику 3 с граничными случаями разворачивания.

Готово, когда: одной командой запускается оценка всего сервиса и выводит таблицу метрик и список ошибок.

Договорённости и открытые вопросы

Самые рискованные места — границы между модулями и противоречие в условии. Их нужно закрыть до начала кода.

Договорённости на границах

- `link_extractor` не разбирает перечисления: «1, 2, 3» передаётся как одна строка, разбирает её `number_parser`.
- `law_matcher` не ищет закон в тексте: он получает уже вырезанный `law_text`.
- Объединяет всё только `detect_links` в `main.py`. Другие модули друг друга не импортируют.
- Изменения `contracts.py` — только по согласию всех пятерых, отдельным PR.

Порядок работы

1. День 1: участник 4 готовит скелет и `contracts.py`, участник 5 начинает разметку, остальные трое пишут свои модули на простых строках.
2. Далее: каждый модуль в своём PR с юнит-тестами.
3. Когда все три модуля готовы: участник 4 собирает сервис, участник 5 запускает оценку и раздаёт ошибки владельцам.
4. Цикл «оценка → правки → оценка», затем Docker-проверка и публикация на GitHub.